Relación Uno A Muchos

Para modelar una relación de **Uno a Muchos (1:N)** como la de un Paciente y sus Reservas (citas médicas) en MongoDB, la decisión depende del **volumen de datos** y de **cómo se consumen** en tu aplicación web.

A diferencia del modelo relacional, donde siempre tendrías una tabla `Paciente` y una tabla `Reserva` con una `FK_pacienteID`, en MongoDB tienes tres caminos principales:

1. Referencia del Padre al Hijo (Array de IDs)

Es la más común si un paciente no tiene miles de reservas (por ejemplo, un historial de 50-100 citas). El documento del paciente guarda un arreglo con los IDs de sus reservas.

Colección `pacientes` **:**

JSON

```
{
  "_id": ObjectId("65f1...a01"),
  "nombre": "Juan Pérez",
  "rut": "12.345.678-9",
  "reservas": [
    ObjectId("99a...01"),
    ObjectId("99a...02")
  ]
}
```

Colección `reservas` **:**

JSON

```
{
  "_id": ObjectId("99a...01"),
  "fecha": "2026-04-10T10:30:00Z",
  "especialidad": "Cardiología",
  "paciente_id": ObjectId("65f1...a01")
}
```

- **Uso en Web:** Ideal para mostrar el "Perfil del Paciente" y listar sus últimas citas rápidamente.

2. Anidamiento (Embedding)

Si las reservas son datos que **siempre** vas a mostrar junto al paciente y no necesitas consultar las reservas de forma aislada (sin el contexto del paciente), puedes meterlas dentro.

Colección `pacientes`:

JSON

```
{
  "_id": ObjectId("65f1...a01"),
  "nombre": "Ana María",
  "historial_reservas": [
    {
      "fecha": "2026-03-15",
      "doctor": "Dr. Soto",
      "estado": "Completada"
    },
    {
      "fecha": "2026-05-20",
      "doctor": "Dra. Rivas",
      "estado": "Pendiente"
    }
  ]
}
```

- **Ventaja:** Velocidad de lectura atómica (una sola consulta trae todo).
- **Riesgo:** Si el paciente llega a tener miles de reservas, el documento podría volverse muy pesado (límite de 16MB por documento en MongoDB).

3. Referencia del Hijo al Padre (Estilo Relacional)

Si esperas que un paciente tenga un historial masivo de reservas (cientos o miles a lo largo de los años), lo mejor es mantenerlas separadas y que solo la reserva apunte al paciente.

Colección `reservas`:

JSON

```
{
  "_id": ObjectId("99a...01"),
  "paciente_id": ObjectId("65f1...a01"), // Solo referenciamos al padre
  "fecha": "2026-04-15",
  "centro_medico": "Sucursal Puente Alto"
}
```

Uso en Web: Para una vista de "Agenda del Día" donde el administrativo ve todas las reservas de hoy de distintos pacientes. Es más fácil filtrar por fecha en una colección de reservas independiente.

¿Cómo elegir para tu App?

Para un sistema médico profesional, te sugiero el **Modelo Híbrido**:

1. Ten las **Reservas** en su propia colección (para que el doctor pueda ver su agenda fácilmente).
2. En la colección de **Pacientes**, guarda solo las **3 últimas reservas** anidadas (desnormalizadas).

¿Por qué? Porque cuando el recepcionista busca al paciente en la web, la ficha carga instantáneamente con las citas más recientes sin tener que hacer un "join" o una segunda consulta a la tabla de reservas.